

Ejercicio 1

Análisis del Programa

Entrada: una cadena de texto "numero" que representa un comprobante electrónico.

Salida: un valor booleano (true / false) que indica si el comprobante es válido.

Reglas de negocio a validar:

- El comprobante debe tener exactamente 13 caracteres, con el formato YXXX-XXXXXXX.
- Y (primer carácter) debe ser 'B' (Boleta) o 'F' (Factura).
- Los 3 caracteres siguientes a Y forman, junto con Y, la serie de 4 caracteres (ej. F001).
- El carácter en la posición 5 debe ser el guion '-'.
- Los 8 caracteres finales (correlativo) deben ser todos dígitos.

Diagrama de Flujo

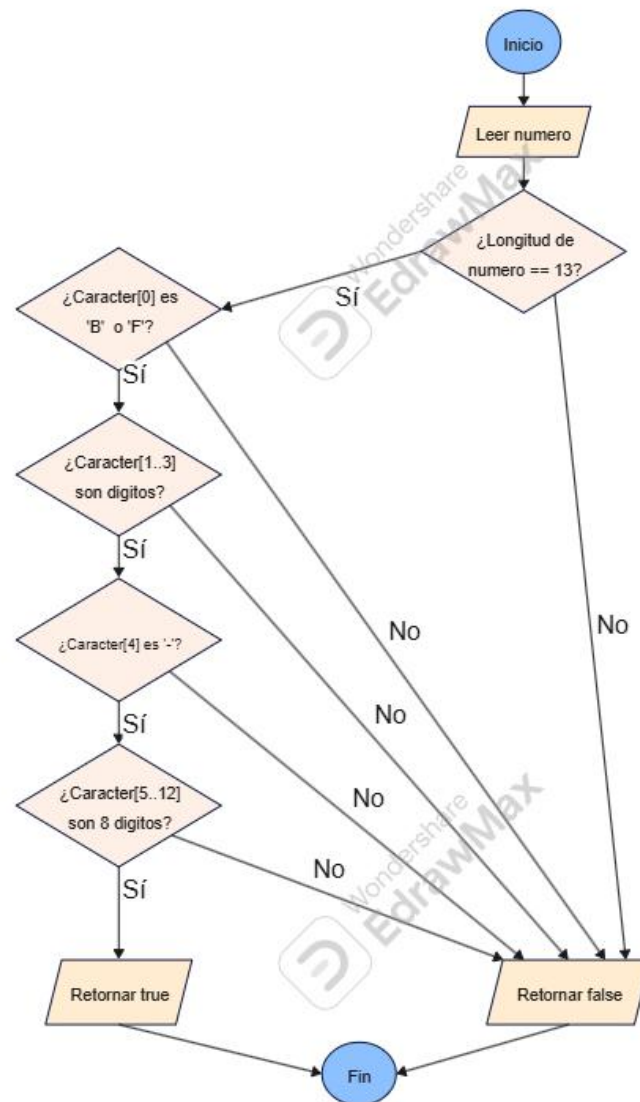


Diagrama de Flujo

```
Inicio
Leer numero

Si longitud(numero) != 13 entonces
    Retornar false
FinSi

Si numero[0] != 'B' Y numero[0] != 'F' entonces
    Retornar false
FinSi

Si NO(numero[1] es digito Y numero[2] es digito Y numero[3] es digito) entonces
    Retornar false
FinSi

Si numero[4] != '-' entonces
    Retornar false
FinSi

Si NO(numero[5..12] son 8 digitos) entonces
    Retornar false
FinSi

Retornar true
Fin
```

Codificación en C#

```
using System;
using System.Text.RegularExpressions;

public class ValidadorComprobante
{
    public static bool ValidarComprobanteElectronico(string numero)
    {
        if (string.IsNullOrEmpty(numero))
            return false;

        // Formato esperado: YYYY-XXXXXXXX
        // Y -> 'B' (Boleta) o 'F' (Factura)
        // XXX -> 3 digitos que completan la serie de 4 caracteres
        // XXXXXXXX -> 8 digitos del correlativo
        string patron = @"^[BF]\d{3}-\d{8}$";

        return Regex.IsMatch(numero, patron);
    }

    public static void Main()
    {
        Console.WriteLine(ValidarComprobanteElectronico("B123-00032123")); // true
        Console.WriteLine(ValidarComprobanteElectronico("A121-32121")); // false
    }
}
```

Ejercicio 2

Análisis del Programa

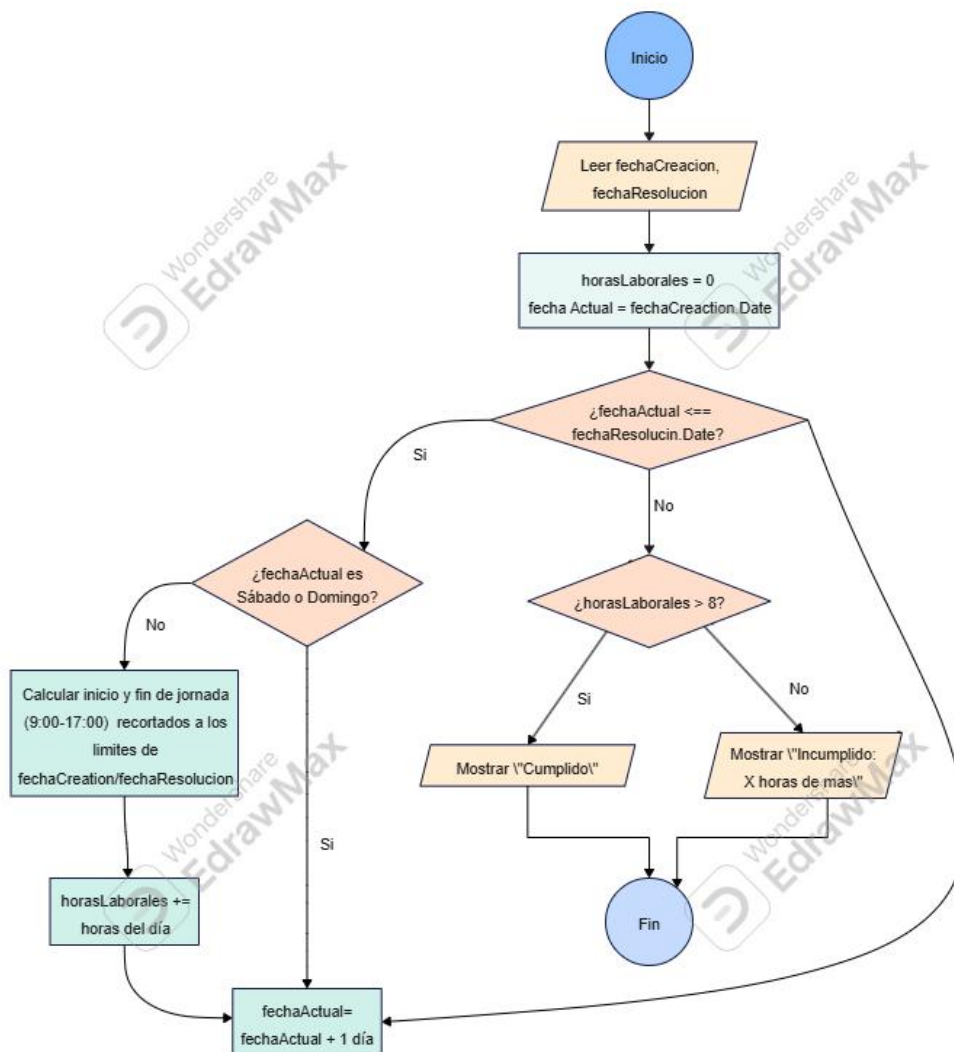
Entrada: fechaCreacion y fechaResolucion (tipo DateTime), que incluyen fecha y hora.

Salida: el estado del SLA como texto: "Cumplido" o "Incumplido: X horas de más".

Reglas de negocio:

- La jornada laboral va de 9:00 a.m. a 5:00 p.m. (8 horas por día).
- No se cuentan sábados ni domingos.
- Se deben sumar solo las horas laborales transcurridas entre la creación y la resolución del ticket.
- Si el total de horas laborales es menor a 8, el SLA está "Cumplido".
- Si el total es igual o mayor a 8, el SLA está "Incumplido", indicando cuántas horas de más se usaron.

Diagrama de Flujo



Pseudocódigo

```
Inicio
  Leer fechaCreacion, fechaResolucion
  horasLaborales <- 0
  fechaActual <- Fecha(fechaCreacion)

  Mientras fechaActual <= Fecha(fechaResolucion) hacer
    Si fechaActual no es Sabado Y fechaActual no es Domingo entonces
      inicioJornada <- fechaActual + 09:00
      finJornada    <- fechaActual + 17:00

      Si fechaActual == Fecha(fechaCreacion) Y fechaCreacion > inicioJornada entonces
        inicioJornada <- fechaCreacion
      FinSi

      Si fechaActual == Fecha(fechaResolucion) Y fechaResolucion < finJornada entonces
        finJornada <- fechaResolucion
      FinSi

      Si finJornada > inicioJornada entonces
        horasLaborales <- horasLaborales + (finJornada - inicioJornada)
      FinSi
    FinSi
    fechaActual <- fechaActual + 1 dia
  FinMientras

  Si horasLaborales < 8 entonces
    Escribir "Cumplido"
  SiNo
    Escribir "Incumplido: " + (horasLaborales - 8) + " horas de mas"
  FinSi
Fin
```

Codificación en C#

```
using System;

public class CalculadoraSLA
{
    static readonly TimeSpan HoraInicio = new TimeSpan(9, 0, 0);
    static readonly TimeSpan HoraFin    = new TimeSpan(17, 0, 0);
    const double LimiteSLA = 8.0;

    public static double CalcularHorasLaborales(DateTime fechaCreacion, DateTime
fechaResolucion)
    {
        double horasLaborales = 0;
        DateTime fechaActual = fechaCreacion.Date;

        while (fechaActual <= fechaResolucion.Date)
        {
            if (fechaActual.DayOfWeek != DayOfWeek.Saturday &&
                fechaActual.DayOfWeek != DayOfWeek.Sunday)
            {
                DateTime inicioJornada = fechaActual + HoraInicio;
                DateTime finJornada    = fechaActual + HoraFin;

                if (fechaActual == fechaCreacion.Date && fechaCreacion > inicioJornada)
                    inicioJornada = fechaCreacion;

                if (fechaActual == fechaResolucion.Date && fechaResolucion < finJornada)
                    finJornada = fechaResolucion;

                if (finJornada > inicioJornada)
                    horasLaborales += (finJornada - inicioJornada).TotalHours;
            }

            fechaActual = fechaActual.AddDays(1);
        }

        return horasLaborales;
    }

    public static string EvaluarSLA(DateTime fechaCreacion, DateTime fechaResolucion)
    {
        double horas = CalcularHorasLaborales(fechaCreacion, fechaResolucion);

        if (horas < LimiteSLA)
            return "Cumplido";
        else
            return $"Incumplido: {horas - LimiteSLA} horas de mas";
    }

    public static void Main()
    {
        DateTime creacion    = new DateTime(2023, 10, 10, 14, 0, 0);
        DateTime resolucion = new DateTime(2023, 10, 11, 10, 0, 0);

        Console.WriteLine($"Horas laborales: {CalcularHorasLaborales(creacion, resolucion)}");
        Console.WriteLine(EvaluarSLA(creacion, resolucion));
    }
}
```